The Software Testing Life Cycle

Six phases, each with its own entry gate, exit gate, activities, and deliverables.

UNIVERSITY QUESTION · MU — MAY 16, DEC 17, MAY 18, DEC 18

"Explain STLC in detail." — asked four times in five years. Learn this one cold.

1.6 · DEFINITION

"Software testing is not a single, isolated activity. It is a series of activities carried out methodically to help certify your software product."

The Software Testing Life Cycle (STLC) is a sequence of specific activities conducted during the testing process to ensure software quality goals are met. It involves both verification activities (are we building the product right?) and validation activities (are we building the right product?).

testing = one step near the end.
Reality: testing = its own six-phase life cycle, running alongside development.

The Six Phases of STLC

A strict sequence — each phase gated by entry and exit criteria.

1 **Requirement Analysis** → 2 **Test Planning** → 3 **Test Case Development** → 4 **Environment Setup** → 5 **Test Execution** → 6 **Test Cycle Closure**

EVERY ARROW IS A GATE

Each stage has a definite entry criteria and exit criteria, plus its own activities and deliverables. In an ideal world, you do not enter the next stage until the exit criteria of the previous one are met — though in practice, phases often overlap under deadline pressure.

THE GATES

Entry & Exit Criteria

ENTRY CRITERIA

The prerequisite items that must be completed before testing can begin.

Think of it as the bouncer at the door: “you don't get in without these items in hand.”

EXIT CRITERIA

The items that must be completed before testing can be concluded.

The graduation requirement: “you don't leave this phase until these boxes are checked.”

Every level of the STLC has both. Ideally, you never enter the next stage until the previous stage's exit criteria are met — but practically, this isn't always possible, which is why we focus on each phase's activities and deliverables.

Requirement Analysis

1

The test team studies the requirements from a testing point of view to identify testable requirements — interacting with stakeholders to understand both functional and non-functional requirements in detail. Automation feasibility for the project is also assessed here.

ACTIVITIES

1. Identify the types of tests to be performed.
2. Gather details about testing priorities and focus.
3. Prepare the Requirement Traceability Matrix (RTM).
4. Identify test environment details for the testing ahead.
5. Automation feasibility analysis (if required).

DELIVERABLES

- Requirement Traceability Matrix (RTM)
- Automation feasibility report (if applicable)

The RTM maps every requirement to the test cases that will verify it — nothing ships untested, nothing is tested twice by accident.

Test Planning

2

A senior QA manager determines the test plan strategy, along with effort and cost estimates for the project. Resources, test environment, test limitations, and the testing schedule are all decided — and the Test Plan itself is prepared and finalized in this phase.

ACTIVITIES

1. Prepare the test plan / strategy document for the various types of testing.
2. Select the testing tools.
3. Estimate the test effort.
4. Plan resources; determine roles and responsibilities.
5. Identify training requirements.

DELIVERABLES

- Test plan / strategy document
- Effort estimation document

This is the phase where testing stops being an intention and becomes a budgeted, scheduled commitment.

Test Case Development

3

With the test plan ready, this phase involves the creation, verification, and rework of test cases and test scripts. Test data is identified, created, reviewed, and reworked based on preconditions — then the QA team develops test cases for individual units.

ACTIVITIES

1. Create test cases and automation scripts (if applicable).
2. Review and baseline the test cases and scripts.
3. Create test data (if the test environment is available).

DELIVERABLES

- Test cases / scripts
- Test data

"Baseline" means the reviewed version is frozen as the official reference — later changes must go through control, not casual edits.

Test Environment Setup

4

This phase decides the software and hardware conditions under which the product is tested. It's one of the most critical aspects of the process — and it can run in parallel with Test Case Development. If the development team provides the environment, the test team's job is a readiness check: the smoke test.

ACTIVITIES

1. Understand the required architecture and environment set-up; prepare the hardware and software requirement list.
2. Set up the test environment and test data.
3. Perform a smoke test on the build.

DELIVERABLES

- Environment ready, with test data set up
- Smoke test results

A smoke test asks one question: "is this build stable enough to even bother testing seriously?" — named after hardware engineering, where you power on a circuit and check for smoke.

Test Execution

5

Testers carry out testing of the software build based on the test plans and test cases prepared earlier. The work consists of test script execution, script maintenance, and bug reporting. Reported bugs go back to the development team for correction — and then retesting.

ACTIVITIES

1. Execute tests as per the plan.
2. Document test results; log defects for failed cases.
3. Map defects to test cases in the RTM.
4. Retest the defect fixes.
5. Track each defect to closure.

DELIVERABLES

- Completed RTM with execution status
- Test cases updated with results
- Defect reports

Test Cycle Closure

6

The completion of test execution — test completion reporting, collection of test completion metrics, and results. The team meets to analyze the testing artifacts, extract lessons from the current cycle, and identify strategies to remove process bottlenecks from future cycles.

ACTIVITIES

1. Evaluate cycle completion criteria: time, test coverage, cost, software, critical business objectives, quality.
2. Prepare test metrics based on those parameters.
3. Document the learning out of the project; prepare the Test Closure Report.
4. Report quality of the work product to the customer — qualitatively and quantitatively.
5. Analyze results to find defect distribution by type and severity.

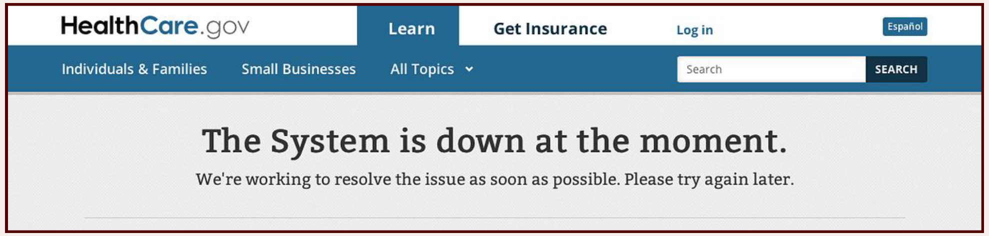
DELIVERABLES

- Test Closure Report
- Test metrics

CASE STUDY

Healthcare.gov, 2013

A launch that compressed an entire STLC into two weeks — and paid for it in public.



On October 1, 2013, the United States launched Healthcare.gov — the official website where millions of Americans would shop for health insurance under a landmark new law. It was one of the most anticipated software launches in government history: 55 contractors, five government agencies, around 300 insurers, all integrating into one system, with a legally mandated, immovable launch date.

THE STLC FAILURE AT THE CENTRE OF THIS CASE

Contractors testified to Congress that full end-to-end testing — which they said needed months was squeezed into the final two weeks before launch. It failed. The site launched anyway.

6

people successfully enrolled on day one — out of over 4 million visitors

TIME TO FIRST CRASH	E2E TESTING WINDOW
~2 hours	**~2 weeks**

Insurers who trialled the system a month early advised against launching. A pre-launch simulation of a few hundred concurrent users flatlined the site. On launch day, roughly 250,000 arrived.

STLC AUTOPSY

P1 Requirements kept changing — a late decision forced users to register accounts before browsing, multiplying load.

P2 No single owner: no senior executive was designated to integrate 55 contractors' work.

P4 Capacity planning missed badly — the environment was sized for a fraction of real traffic.

P5 Execution compressed to two weeks; the failed results were overridden by the immovable date.

Four Transferable Lessons

LESSON 01

Exit criteria exist to be obeyed

End-to-end testing failed its exit criteria — and the launch proceeded anyway. A gate you can override under pressure is not a gate.

LESSON 02

Environment setup is a real phase, not a checkbox

The site was sized for a fraction of its real load. Phase 4 skipped is Phase 5 doomed.

LESSON 03

Late requirement changes ripple through every phase

One late decision — register before browsing — invalidated earlier capacity assumptions and test plans.

LESSON 04

The STLC survives contact with politics only if someone owns it

With 55 contractors and no designated integrator, nobody was accountable for the cycle as a whole.